Interface contract

Status. Normative. Every module in the system is built against the interfaces, file formats and invariants in this document. Where a knowledge pack gives the algorithm, this document gives the seams between algorithms. When the thesis and this document disagree on an interface, this document wins; when a pack and this document disagree on a fact from a paper, the pack wins.

Scope. Repository layout; the on-disk formats for every store (trajectories, Notebook, skills, library index, gate log, benchmark, LoRA archive, sleep reports); the HTTP contract to the model servers; the TypeScript interfaces each module implements; the event flow of one task; the invariants that protect integrity; versioning and migration. Labelled defaults cite the pack (A–I) and section they come from; unlabelled values are design choices to be tuned by the benchmark.

---

1. Principles that shape every interface

1. **Plain files under git.** Every store is a directory of JSON, JSONL or Markdown that a person can read and `git diff`. No opaque databases except a rebuildable search index. (Thesis 5.8; Pack A §2.3.)
2. **Verifier is the only source of truth.** Nothing enters skills, Notebook lessons or training data without a `VerifyResult.ok === true` from the cascade. The verifier directory is owned by the human (invariant I-1).
3. **Deltas, not rewrites.** The Notebook and the skill library change by appending operations; a full rewrite of a store is never performed by an agent. (Pack A: ACE grow-and-refine; context collapse.)
4. **Every model call is logged.** Prompts, responses, token counts and timings go to the trajectory of the task that caused them. Training data is a view over trajectories, never a separate hand-made set.
5. **Slots, not models.** Code talks to `worker`, `senior`, `draft`, `embed`, `rerank` — never to a model name. A slot is an HTTP base URL plus limits.
6. **Everything has an id, a timestamp and a provenance.** Provenance means the task id and trajectory that produced it.
7. **Fail closed.** A missing verifier command for a stage is a *skipped* stage reported as such, never a pass. A gate that cannot decide routes to the slow path only if budget allows, otherwise asks.

---

2. Repository layout

```
tfa/                            ← the tool (TypeScript, Node ≥ 20)
  package.json  tsconfig.json  tfa.config.json
  src/
    config.ts                    slots, cascade, budgets, paths
    cli.ts                       tfa doctor | verify | fix | task | bench | sleep | train | models
    llm/        client.ts  edits.ts  prompts/           model I/O
    gate/       gate.ts  features.ts  calibrate.ts
    retrieve/   index.ts  chunk.ts  hybrid.ts  rerank.ts  versions.ts
    think/      search.ts  plan.ts  candidates.ts  stop.ts
    verify/     cascade.ts  detect.ts  exec.ts  stages/   ← HUMAN-OWNED (I-1)
    memory/     notebook.ts  reflector.ts  curator.ts  core.ts
    skills/     store.ts  match.ts  compose.ts  save.ts
    sleep/      consolidate.ts  decay.ts  prethink.ts  report.ts
```

```
    train/      dataset.ts  sft.ts  dpo.ts  archive.ts  (calls Python)
    bench/      run.ts  tasks.ts  holdout.ts  report.ts  ← HUMAN-OWNED (I-1)
    log.ts      trajectory writer
  python/                       training only: qlora_sft.py  dpo.py  merge_eval.py
  test/                         vitest; fixture repos; mock model server
  docs/                         thesis, packs, this contract, build prompt

<target-repo>/.tfa/             ← per-project state, committed to THAT repo's git
  config.json                   project overrides
  trajectories/<taskId>.jsonl
  notebook/
    core.md                     the always-in-context block (≤ 2k tokens)
    notes.jsonl                 append-only note operations
    index.json                  rebuilt view: current notes by id
  skills/<skillId>/
    skill.json  code.ts  test.ts  README.md
  skills/index.jsonl            append-only skill operations
  gate/decisions.jsonl
  bench/tasks/<taskId>.json     human-owned
  bench/holdout/<taskId>.json   human-owned; never read by train/
  bench/runs/<runId>.json
  lora/archive.jsonl  lora/<loraId>/  (adapter files, score, dataset manifest)
  sleep/reports/<date>.md
  library/index.sqlite          rebuildable; gitignored
  library/manifest.json         what is indexed, with content hashes and versions
```

Human-owned directories are enforced by invariant I-1 (section 9). Agents may read them and may propose changes as patches in `proposals/`, never write them directly.

---

3. Identifiers, time, provenance

```ts
        type Id = string;                 // `${kind}_${base32(time)}${rand4}` e.g. task_01HV…, note_…, skill_…, cand_…, lora_…
        type Ms = number;                 // Unix epoch milliseconds, UTC
        type Hash = string;               // sha256 hex of file content or canonical JSON
        type Provenance = { taskId: Id; trajectory: string; attempt?: number; candidateId?: Id; at: Ms };
```

Ids are globally unique and sortable by time. Every record in every store carries `id`, `at` and, where it was produced by a task, `prov: Provenance`.

---

4. Model server contract

All five slots speak the OpenAI-compatible HTTP API served by `llama-server`. The orchestration layer never depends on anything else.

```ts
        type Slot = "worker" | "senior" | "draft" | "embed" | "rerank";
        type SlotConfig = { url: string; maxTokens: number; temperature: number; ctx: number; family: string; resident: boolean };
```

Endpoint	Used by	Required fields	Notes
`POST /v1/chat/completions`	worker, senior	`messages`, `max_tokens`, `temperature`, `stream:false`, `cache_prompt:true`, `id_slot`	`cache_prompt` reuses the KV prefix; `id_slot` pins the system+core block to one server slot (Pack G §2: prefix caching).

Endpoint	Used by	Required fields	Notes
`POST /v1/embeddings`	embed	`input: string[]`	Batch ≤ 64; cache by `Hash` of input (Pack G: content-hash caches).
`POST /rerank` (llama.cpp) or local cross-encoder	rerank	`query`, `documents[]`, `top_n`	Returns `{index, relevance_score}[]`.
`GET /health`	all	—	Used by `tfa doctor` and by the senior loader.
`POST /slots/{id}?action=save|restore`	worker	`filename`	Persist the cached prefix across restarts.

Prompt structure (fixed order, so the prefix caches): 1. system prompt (static per module) 2. `core.md` (Notebook core block, ≤ 2,000 tokens) 3. retrieved context (budgeted; see §6.3) 4. task + conversation

Senior lifecycle. The senior is started on demand by `think/` when the gate routes slow or after two fast-path failures, kept warm for `senior.idleUnloadMs` (default 10 min), and stopped by `sleep/` before training to free RAM. Speculative decoding (worker as draft for a same-family senior) is a server flag, invisible to this contract.

Edit format. Models produce edits as a JSON array of `{path, find, replace}` or `{path, content}`; `find` must match exactly once; paths are repo-relative and may not escape the repo. A parse or apply failure is cascade stage `apply` (always first, cost ≈ 0).

---

5. Trajectories — `trajectories/<taskId>.jsonl`

One JSON object per line; `t` is the event type. This is the system's raw memory and the sole source for training data.

```
        type Ev =
          | { t:"task";      id:Id; at:Ms; repo:string; text:string;
origin:"user"|"queue"|"bench"|"sleep" }
          | { t:"decompose"; at:Ms; subtasks:{ id:Id; text:string; touches:string[] }[] }
          | { t:"gate";      at:Ms; subtaskId:Id; decision:"fast"|"slow"|"ask";
features:GateFeatures; score:number; threshold:number }
          | { t:"retrieve";  at:Ms; subtaskId:Id; query:string; hits:{ chunkId:Id; score:number;
cited:boolean }[]; tokens:number }
          | { t:"prompt";    at:Ms; subtaskId:Id; attempt:number; slot:Slot; messages:Msg[];
promptHash:Hash }
          | { t:"response";  at:Ms; subtaskId:Id; attempt:number; slot:Slot; text:string;
ms:number; promptTokens?:number; completionTokens?:number }
          | { t:"candidate"; at:Ms; subtaskId:Id; candidateId:Id; planId?:Id; parentId?:Id;
edits:Edit[] }
          | { t:"stage";     at:Ms; candidateId:Id; stage:Stage; ok:boolean; skipped?:boolean;
ms:number; log:string }
          | { t:"verify";    at:Ms; candidateId:Id; ok:boolean; failedAt?:Stage; totalMs:number }
          | { t:"reflect";   at:Ms; subtaskId:Id; lessons:LessonDelta[]; playbook:PlaybookDelta[]
}
          | { t:"save";      at:Ms; subtaskId:Id; skillId?:Id; noteOps:Id[]; trainingExample?:Id;
preferencePair?:Id }
          | { t:"ask";       at:Ms; subtaskId:Id; question:string; answeredAt?:Ms; answer?:string
}
          | { t:"correction";at:Ms; subtaskId:Id; candidateId:Id; humanDiff:string }      // human
edited the result
          | { t:"result";    at:Ms; ok:boolean; subtaskId?:Id; candidateId?:Id; diff?:string;
note?:string; costs:Costs };
```

```
        type Costs = { wallMs:number; promptTokens:number; completionTokens:number;
seniorCalls:number; stagesRun:number; wh?:number };
```

Rules: a trajectory is append-only; `log` fields are tails (≤ 6 kB, ANSI stripped); a `correction` event is written whenever a human changes a file the system changed within 24 h (detected by git diff against the saved candidate).

---

6. Stores

6.1 Notebook — `notebook/notes.jsonl` + `notebook/core.md`

Operations only; `index.json` is a rebuilt view. Format and fields follow Pack A (ACE bullets with helpful/harmful counters; A-MEM links; Generative-Agents-style scoring).

```
        type NoteKind = "fact" | "lesson" | "playbook";
        type Note = {
          id:Id; kind:NoteKind; section:string;            // section = ACE playbook section
name, e.g. "nextjs.app-router"
          text:string;                                      // one atomic statement, ≤ 400 chars
          tags:string[]; links:Id[];                        // A-MEM: links to related notes
          helpful:number; harmful:number;                   // ACE counters, updated by curator
from verify outcomes
          confidence:number;                                // 0..1, derived:
helpful/(helpful+harmful+1)
          uses:number; lastUsed:Ms; createdAt:Ms; prov:Provenance;
          test?:{ kind:"assert"|"vitest"|"grep"; path:string }; // lessons-as-tests: compiled
check, if any
          retired?:Ms;
        };
        type NoteOp =
          | { op:"add";     at:Ms; note:Note }
          | { op:"count";   at:Ms; id:Id; helpful?:1; harmful?:1; prov:Provenance }
          | { op:"link";    at:Ms; id:Id; to:Id }
          | { op:"merge";   at:Ms; into:Id; from:Id[]; text:string; by:"sleep" }
          | { op:"retire";  at:Ms; id:Id; reason:string; by:"sleep"|"human" }
          | { op:"promote"; at:Ms; id:Id; toCore:boolean; by:"sleep" };
```

Core block (`core.md`): rendered by `sleep/` from notes with `promote toCore=true`, ordered by utility `uses × confidence`, compacted to ≤ 2,000 tokens (Pack A §2.4). It is text, included verbatim as prompt section 2.

Retrieval of notes for a subtask: score = recency + importance + relevance with equal weights, recency = 0.995^h where h = hours since last use (Pack A: Generative Agents); top-k by score within the context budget; every retrieved note gets `uses++` and `lastUsed`.

Delta discipline: the curator may emit `add`, `count`, `link` only. `merge`, `retire`, `promote` are sleep-only. No op ever edits `text` in place except `merge`, which creates a new text and retires the sources.

6.2 Skills — `skills/<skillId>/` + `skills/index.jsonl`

```
        type Param = { name:string; type:string; description:string; required:boolean };
        type Skill = {
          id:Id; name:string; version:number; description:string;     // description is what gets
embedded (Voyager: Pack B)
          inputs:Param[]; outputs:Param[];                           // the contract
          code:"code.ts"; test:"test.ts";                           // relative files; test
must pass in a clean clone
          deps:{ pkg:string; range:string }[];                      // dependency versions it
was verified against
```

```
          tags:string[]; embedding:Hash;                                  // embedding cached by
hash of description
          successes:number; failures:number; lastUsed:Ms; createdAt:Ms; prov:Provenance;
          retired?:Ms; supersededBy?:Id;
        };
        type SkillOp =
         | { op:"add"; at:Ms; skill:Skill }
         | { op:"use"; at:Ms; id:Id; ok:boolean; prov:Provenance }
         | { op:"retire"; at:Ms; id:Id; reason:string; supersededBy?:Id; by:"sleep"|"human" };
```

Save rule (`skills/save.ts`): a candidate becomes a skill only if (1) its cascade passed, (2) a second run from a clean `git worktree` passes again, (3) the worker produced a parameterized version + test that pass, (4) no existing skill has embedding similarity ≥ 0.92 with the same `touches` footprint (else `use` on the existing one). Save = write folder + `add` op.

Match rule (`skills/match.ts`): candidates = top-k by cosine similarity of the subtask text to skill descriptions (k=5), filtered by `deps` compatible with the target repo's `package.json` ranges; a match is "covered" if similarity ≥ 0.80 and the skill's test passes in the target context.

6.3 Library — `library/index.sqlite` + `library/manifest.json`

Rebuildable; never a source of truth.

```
        type Doc = { id:Id; path:string; kind:"code"|"doc"|"note"; pkg?:string; version?:string;
hash:Hash; indexedAt:Ms };
        type Chunk = { id:Id; docId:Id; kind:"function"|"component"|"export"|"section"|"text";
symbol?:string;
                       start:number; end:number; text:string; hash:Hash; embedding:Float32Array
};
        type SymbolRow = { symbol:string; docId:Id; chunkId:Id; kind:string };          // exact-
match index
        type Query = { text:string; symbols?:string[]; versions:Record<string,string>;
budgetTokens:number };
        type Hit = { chunkId:Id; score:number; lexical:number; dense:number; rerank?:number };
```

Pipeline (Pack G §2): chunk by syntax unit for code and by heading for docs; lexical (BM25/FTS5) ∪ dense (embed slot) → reciprocal-rank fusion → rerank top-40 → pack under `budgetTokens` with hard cut; **version filter first**: documentation chunks whose `version` does not satisfy the repo's dependency range are excluded before scoring (thesis 5.5). Every chunk used in a prompt is recorded as a `retrieve` hit with `cited:true` if the response referenced it.

6.4 Gate log — `gate/decisions.jsonl`

```
        type GateFeatures = {
          simTask:number;            // max cosine similarity to solved tasks (trajectories with
result.ok)
          simSkill:number;           // max similarity to skill descriptions
          skillCovered:boolean;      // match rule satisfied
          footprintKnown:number;     // fraction of touched files/packages seen in past successes
          selfEstimate:number;       // worker one-token P("known")
          size:number;               // estimated edit size (lines) from decomposition
        };
        type GateDecision = { id:Id; at:Ms; taskId:Id; subtaskId:Id; features:GateFeatures;
score:number; threshold:number;
                              decision:"fast"|"slow"|"ask"; outcome?:{ ok:boolean;
escalated:boolean; wallMs:number; cost:Costs } };
```

Policy (Pack H §2): score = logistic regression over features, refit by `gate/calibrate.ts` from decisions with outcomes; threshold chosen with asymmetric costs (fast-path error ≈ seconds, slow-

path error ≈ hour) and a daily slow-path budget; cold start = rule: `skillCovered || simTask ≥ 0.85 → fast`, else slow.

6.5 Benchmark — `bench/tasks/*.json`, `bench/holdout/*.json`, `bench/runs/*.json`

Human-owned task files (I-1). Format mirrors SWE-bench (Pack H).

```
        type BenchTask = { id:Id; repo:string; baseCommit:string; text:string;
difficulty:"routine"|"moderate"|"hard";
                           failToPass:string[]; passToPass:string[]; setup?:string;
timeoutMs:number; createdAt:Ms; retiredAt?:Ms };
        type BenchRun = { id:Id; at:Ms; set:"dev"|"holdout"; config:Hash; loraId?:Id; results:{
taskId:Id; ok:boolean; attempts:number;
                           firstTry:boolean; corrections:number; costs:Costs }[]; summary:{
passRate:number; firstTryRate:number; meanCost:Costs } };
```

Protocol: 20 dev + 10 holdout; holdout never read by `train/` or `sleep/` (enforced by path, I-2); rotated monthly: retire holdout → dev, mine 10 fresh tasks postdating the latest LoRA's dataset cut; a 0.95 similarity check against training trajectories rejects contaminated tasks (Pack H §2).

6.6 LoRA archive — `lora/archive.jsonl` + `lora/<loraId>/`

```
        type LoraEntry = { id:Id; at:Ms; base:string; domain:string; r:number; alpha:number;
method:"sft"|"sft+dpo";
                           dataset:{ manifest:string; examples:number; pairs:number; cutAt:Ms;
replayFraction:number };
                           holdout:{ runId:Id; passRate:number; firstTryRate:number }; baseline:{
runId:Id; passRate:number };
                           active:boolean; rejected?:string };
```

Rules: train the worker only; one entry per weekly run; `active` flips only if `holdout.passRate ≥ baseline.passRate + 0.02` and no off-domain regression > 0.05 (both thresholds are design defaults); never merged into base weights; adapter files are never deleted.

6.7 Training views — `train/dataset.ts`

Derived, never hand-edited; rebuilt from trajectories at each weekly run.

```
        type SftExample = { id:Id; prov:Provenance; messages:Msg[]; completion:string;
reasoning?:string; stages:Stage[] };   // ok candidates only
        type PrefPair  = { id:Id; prov:Provenance; prompt:Msg[]; chosen:string; rejected:string;
source:"verified-vs-failed"|"human-correction" };
```

Defaults from Pack D: QLoRA NF4 base, double quantization, paged AdamW; rank/alpha and learning rates as the pack's table; DPO β as the pack's default; replay fraction ≥ 0.2 from earlier weeks.

6.8 Sleep reports — `sleep/reports/<date>.md`

Human-readable; one per run: counts of replayed, merged, retired, promoted; flagged contradictions (with note ids); pre-thought queue tasks; training outcome; what needs a human decision.

---

7. Module interfaces

```
        interface Reader   { chat(slot:Slot, messages:Msg[], o?:ChatOpts):Promise<ChatResult>;
embed(texts:string[]):Promise<Float32Array[]>; rerank(q:string, docs:string[]):Promise<number[]>;
healthy(slot:Slot):Promise<boolean>; }
        interface Decomposer { run(task:Task):Promise<Subtask[]> }
```

```
// uses worker; senior for size>threshold
        interface Gate     { decide(sub:Subtask, ctx:GateContext):Promise<GateDecision>;
record(d:GateDecision, outcome:Outcome):void; refit():Promise<void>; }
        interface Retriever{ query(q:Query):Promise<{ hits:Hit[]; packed:string; tokens:number
}>; reindex(paths?:string[]):Promise<void>; }
        interface SkillStore { match(sub:Subtask, repo:RepoInfo):Promise<SkillMatch[]>;
apply(m:SkillMatch, repo:string):Promise<Candidate>; save(c:Candidate,
traj:Trajectory):Promise<Skill|null>; use(id:Id, ok:boolean, prov:Provenance):void; }
        interface Thinker  { search(sub:Subtask, budget:Budget, deps:{ reader:Reader;
retriever:Retriever; verifier:Verifier; notebook:Notebook }):Promise<SearchResult>; }
        interface Verifier { stages(repo:string):StageCommand[]; run(c:Candidate, repo:string,
o?:{ upTo?:Stage }):Promise<VerifyResult>; }   // HUMAN-OWNED
        interface Notebook { retrieve(sub:Subtask, budgetTokens:number):Promise<Note[]>;
core():string; apply(ops:NoteOp[]):void; }
        interface Reflector{ reflect(traj:Trajectory, result:SearchResult):Promise<{
lessons:LessonDelta[]; playbook:PlaybookDelta[] }> }
        interface Curator  { curate(d:ReflectorOutput, notebook:Notebook):NoteOp[] }
// deterministic merge; add/count/link only
        interface Sleeper  { run(budget:{ ms:number; tokens:number }):Promise<SleepReport> }
        interface Trainer  { buildDataset(since:Ms):Promise<DatasetManifest>;
train(m:DatasetManifest):Promise<LoraEntry>; evaluate(e:LoraEntry):Promise<LoraEntry>; }
        interface Bench    { run(set:"dev"|"holdout", cfg:RunConfig):Promise<BenchRun> }
// HUMAN-OWNED
```

Shared types:

```
        type Stage = "apply"|"lint"|"typecheck"|"test"|"build"|"browser"|"citation";
        type Candidate = { id:Id; subtaskId:Id; planId?:Id; parentId?:Id; edits:Edit[];
createdBy:Slot|"skill"; };
        type VerifyResult = { ok:boolean; failedAt?:Stage; stages:{ stage:Stage; ok:boolean;
skipped?:boolean; ms:number; log:string }[]; totalMs:number };
        type Budget = { maxCandidates:number; maxDepth:number; maxWallMs:number;
allowSenior:boolean; stopOnAgreement:boolean };
        type SearchResult = { ok:boolean; best?:Candidate; verify?:VerifyResult; tree:TreeNode[];
costs:Costs; unverified?:boolean };
```

---

8. Event flow of one task (normative sequence)

1. `cli task` → `Ev.task` written; repo must be a git repo with a clean index (else ask).
2. **Decompose** → `Ev.decompose`. If any subtask has `size` > `decompose.seniorThreshold`, decomposition is redone by the senior.
3. For each subtask, in dependency order:
    1. **Gate** → `Ev.gate`. `ask` → `Ev.ask`, wait (interactive) or defer to queue (unattended).
    2. **Retrieve** (notes + library + skills) → `Ev.retrieve`; context packed in the fixed prompt order (§4).
    3. **Fast path:** skill apply → verify. On failure ×2 → escalate to slow path (`gate.decision` stays `fast`, `outcome.escalated=true`).
    4. **Slow path:** `Thinker.search` with `Budget` from difficulty; plan-level sampling; candidates verified by the cascade cheapest-first; expand survivors; stop on agreement or budget (Pack C §2).
    5. **Verify** final candidate end-to-end (all stages, including `citation` for prose) → `Ev.verify`.
    6. **Reflect** → `Ev.reflect`; **Curate** → `NoteOp[]` applied; lessons with a compilable check get a `test` and are added to the cascade's lesson-tests directory (which is human-owned: the system writes a *proposal*, I-1).
    7. **Save** → skill (if save rule passes), training example, preference pair → `Ev.save`.
    8. `Gate.record(outcome)`.
4. `Ev.result` with costs. Edits remain in the working tree for review; the diff is in the trajectory.

5. Within 24 h, `cli` or the watcher detects human edits to touched files → `Ev.correction` → lesson + test proposal + preference pair.

Unattended mode (queue, sleep): identical, except `ask` defers and the senior is allowed full budget.

---

9. Invariants (enforced in code, checked in CI)

Id	Invariant	Enforcement
I-1	Human-owned paths (`src/verify/**`, `src/bench/**`, `.tfa/bench/**`, lesson-tests dir) are never written by any module except through `proposals/<id>.patch`.	A write guard in the file layer rejects writes; CI test asserts the guard; git pre-commit hook on the target repo.
I-2	`train/` and `sleep/` never read `.tfa/bench/holdout/**`.	Path allow-list in those modules; test with a canary file.
I-3	Nothing enters skills, lessons or training views without a `VerifyResult.ok`.	Save functions require the result object, not a boolean.
I-4	A skipped stage is never reported as passed.	`VerifyResult` distinguishes `skipped`; aggregate `ok` ignores skipped but the report shows them; `bench` refuses tasks whose `failToPass` stage is skipped.
I-5	Notebook and skill stores are append-only operation logs.	Writers can only append; `index.json` is rebuilt, never edited.
I-6	Every model call writes `prompt` and `response` events before returning.	Reader wrapper.
I-7	The base model is never modified; LoRAs are never merged into it.	Train scripts have no merge path; archive entries are immutable.
I-8	Edits never escape the target repo; `find` must match exactly once.	`applyEdits` (exists in P0).
I-9	Prompt order is fixed (system, core, context, task).	A single `buildPrompt()`; tests snapshot the order.
I-10	Any constant taken from a paper carries a comment `// arXiv:<id> §<n>`.	Lint rule: numeric literals in `gate/ think/ memory/ train/` need the comment.

---

10. Configuration — `tfa.config.json` (tool) and `.tfa/config.json` (project override)

```
        type Config = {
          slots: Record<Slot, SlotConfig>;
          cascade: Stage[]; timeouts: Partial<Record<Stage, number>>;
          budgets: { fast:{ maxCandidates:1; maxWallMs:60_000 };
slow:Record<"routine"|"moderate"|"hard", Budget>; queue:Budget; contextTokens:number;
coreTokens:number };
          gate: { mode:"rule"|"model"; fastThreshold:number; dailySlowBudgetMs:number };
          skills: { matchThreshold:number; dedupeThreshold:number };
          notebook: { recencyDecay:number /* 0.995, Pack A */; retrieveK:number };
```

```
          sleep: { idleAfterMs:number; maxMs:number; maxTokens:number; decayHalfLifeDays:number
};
          train: { weekday:number; minExamples:number; r:number; alpha:number; lr:number;
dpoBeta:number; replayFraction:number; activateDelta:number };
          bench: { devCount:20; holdoutCount:10; rotateDay:number; contaminationThreshold:0.95 };
          decompose: { seniorThreshold:number };
        };
```

Precedence: project override > tool config > defaults; environment variables override slot URLs only.

---

11. Versioning and migration

- Every store file carries `"schema": <int>` in its first line (JSONL) or frontmatter (Markdown). This document is schema 1.
- Readers accept the current and previous schema; a `tfa migrate` command rewrites stores forward and keeps the original under `.tfa/_migrated/<schema>/`.
- Trajectories are never migrated in place; readers translate on load.
- Changing a type in §5–§7 requires bumping the schema and adding a migration; CI fails otherwise.

---

12. Decisions the human still owns (not in this contract)

1. Senior model A (GLM-4.7-Flash) or B (Qwen3.6-35B-A3B) — settle by P2 benchmark.
2. RAM upgrade to 64 GB — changes `slots.senior` only.
3. The 20 + 10 benchmark tasks.
4. Which target repos are in scope for the first three months.
5. Activation thresholds in §6.6 after the first two weekly runs.